Brainbox — Architecture & Product Overview

High-level flow diagram, SDKs, backend components, tools, and roadmap ideas

Generated: 2026-07-27 13:29 UTC

This document summarizes how the Brainbox system operates, the SDKs available for customers, the backend components (ingestion, AI agents, database, dashboard), and integration points for web, mobile, and Odoo. It is intended as a shareable business-and-technical overview for boards, teams, and developers.

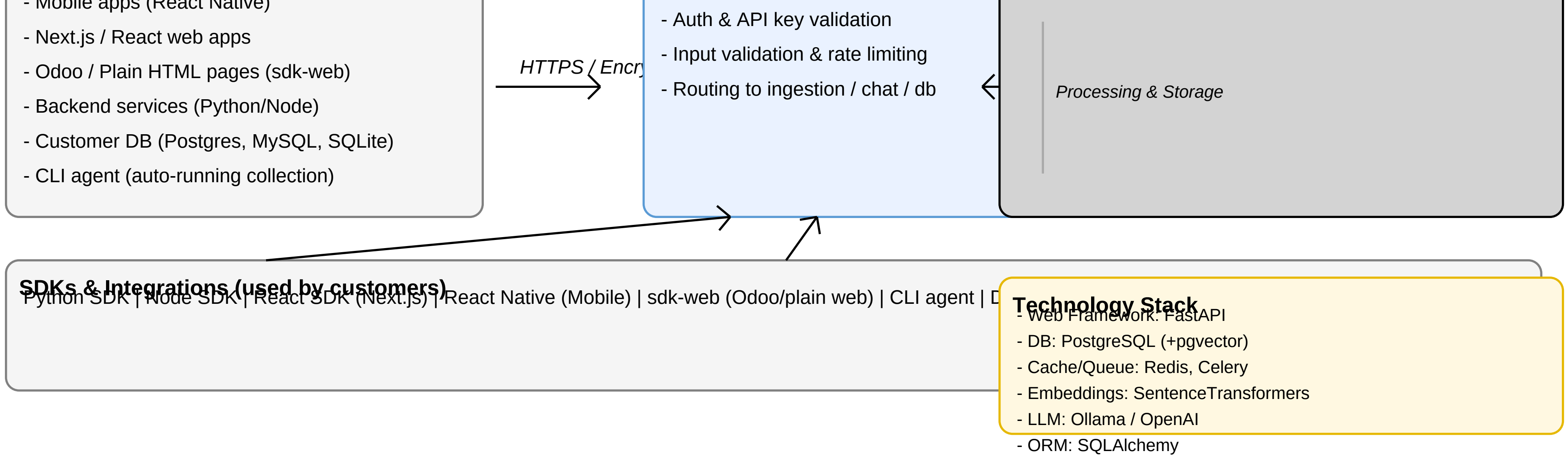

Diagram: high-level flow from customer SDKs → API Gateway → Brainbox backend (ingestion, AI agents, DB) → Dashboard

SDKs, Features & How it Works

SDKs included: Python SDK, Node.js SDK, React SDK (for Next.js), React Native SDK for mobile, sdk-web for plain web and Odoo embedding, CLI agent for automatic log collection, and a Database SDK for safe read-only queries.

How it works (flow): Customer apps and CLI collect logs or send queries using SDKs → Requests go over HTTPS to the API Gateway which handles auth, validation and rate limiting → Ingestion pipeline chunks and deduplicates data, stores embeddings and documents → AI Agents (LangGraph) and LLM interfaces analyze data and produce insights → Results stored in database (+audit logs) and surfaced via Dashboard and Chat endpoints.

Database access: Customers use Database SDK which issues requests to the backend endpoints (/api/db/test-connection, /api/db/schema, /api/db/query, /api/db/audit-log). Backend enforces read-only, rate limits, tenant isolation, and logs all queries for auditing.

Technology Stack & Tools

Core backend: FastAPI (Python), SQLAlchemy ORM, PostgreSQL with pgvector for embeddings, Redis for cache and Celery for background jobs.

Embeddings: SentenceTransformers or other recommended embedding models; vector storage via pgvector. LLMs: Ollama for local inference and OpenAI for cloud inference; configurable per deployment.

SDK & deployment tools: pip, npm for distribution; Docker and docker-compose for deployment; systemd/launchctl/Task Scheduler for running the CLI agent as a service. Monitoring via structured logs, tracing, and audit logs in DB.

Roadmap Ideas & Suggested Features

1) Mobile & Offline: Add offline buffering for mobile SDKs and background sync to handle intermittent connectivity; push notifications for high-severity alerts.

2) Odoo Integration: Provide a ready-made Odoo module that embeds the chat widget and pre-configured endpoints for ingesting logs or business data from Odoo models.

3) Real-time Streaming: Offer optional WebSocket or streaming ingestion for near-real-time observability and alerting.

4) Fine-grained RBAC & Audit: Expand roles and exportable compliance reports; automated data retention rules per tenant.

5) Analytics & Reports: Dashboard widgets for top queries, slow queries, trending errors, and user behavior metrics.

6) Automated Remediation: Add playbooks for common issues and automated suggestions; integrate with incident management tools.

Appendix — Endpoints & Security Highlights

Database endpoints (required): POST /api/db/test-connection, POST /api/db/schema, POST /api/db/query (read-only), GET /api/db/audit-log. Enforce: API key validation, tenant check, block forbidden SQL operations, rate limit, and audit all queries.

Chat & ingest endpoints: POST /api/ingest, POST /api/chat, POST /api/chat/session, health endpoints at /api/health. Ensure HTTPS and API authentication for all endpoints. Use parameterized queries and sanitize outputs to avoid leaking secrets.